Adjoint-based sensitivity using DGM

David A. Buchta

(1 January 2025)

The adjoint state is useful for optimization, sensitivity analysis, data assimilation, experimental design, etc. However, acquiring the solution is costly, since it often requires access to the entire trajectory of the forward state. Practioners cope with this challenge in several ways: i) recomputing the forward state when needed, ii) storing the forward trajectory in memory, or iii) a mixture of these. To mitigate these complications, the forward and adjoint solutions are approximated using the Deep Galerkin Method (**?**). The neural-network weights, for each network, minimize the residual of the forward and adjoint PDEs and their boundary conditions. We demonstrate the approach on an advection-diffusion equation, where the objective is to identify the sensitivity of the solution to initial-condition perturbations. Gradient accuracy for adjoint-forward DGM is compared with standard discretization methods, e.g., finite-difference techniques.

1. Adjoint-Forward Deep Galerkin Method (AFDGM)

Consider the scalar cost function

$$\mathcal{J}(u) = \frac{1}{2}\int_{\mathcal{T}}\int_{\Omega} I(u(x,T))\, d\Omega d\mathcal{T}, \tag{1.1}$$

where the solution at the end of the time period of interest, $u(x,T)$, is acquired by solving the following PDE system

$$\mathcal{M}(u) \equiv \partial_t u(x,t) + \mathcal{D}(u)u(x,t) = 0 \quad (x,t) \in \mathcal{T}\times\Omega \tag{1.2}$$

$$u(x,t=0) = u_0(x) \tag{1.3}$$

$$u(x,t) = b(x,t) \quad x \in \partial\Omega. \tag{1.4}$$

The operator $\mathcal{D}(u)$ is potentially nonlinear; $u_0(x)$ is the initial conditions; and, $b(x,t)$ are the boundary conditions. The objective is to identify earlier state perturbations $\delta u(x,t)$ for $t < T$ that optimizes the cost in (1.1), e.g., initial condition perturbations that increase (or decrease) the cost. One approach is to use the local gradient of the cost to guide the direction of increase, $\delta u(x,t=0) = \beta\nabla\mathcal{J}$. The gradient is obtained by considering the first-order variation of $\mathcal{J}$ and subtracting the constraint of that perturbations of the state $\delta u(x,t)$ must obey the PDE system,

$$\delta\mathcal{J}(u) = \partial_u\mathcal{J}\delta u - \int_{\mathcal{T}}\int_{\Omega}(u^\dagger\partial\mathcal{M}_u\delta u)\, d\Omega d\mathcal{T}. \tag{1.5}$$

Substituting the PDE, the change in the cost is

$$\delta\mathcal{J}(u) = \int_{\Omega}\partial_u I(u)\delta u(x,T)\, d\Omega - \int_{\mathcal{T}}\int_{\Omega} u^\dagger\left[\partial_t\delta u(x,t) + \partial_u\mathcal{D}\delta u(x,t)\right] d\Omega d\mathcal{T}, \tag{1.6}$$

and integrating by parts

$$\begin{aligned}\delta\mathcal{J}(u) = &\int_{\Omega} u^{\dagger}(x,0)\delta u(x,0)\, d\Omega \\ &+ \int_{\Omega} \partial_u I(u)\delta u(x,T) - u^{\dagger}(x,T)\delta u(x,T)\, d\Omega \\ &- \int_{\mathcal{T}}\int_{\Omega} \delta u \left[\partial_t u^{\dagger}(x,t) + \mathcal{D}^{\dagger}(u)u^{\dagger}(x,t)\right]\, d\Omega d\mathcal{T} - B(u^{\dagger},\delta u),\end{aligned} \tag{1.7}$$

where $B(u^{\dagger},\delta u)$ represent spatial boundary terms which depend on the form of $\mathcal{D}$, isolates state perturbations δu from within the evolution of the PDE. The following adjoint PDE system

$$\mathcal{M}^{\dagger}(u,u^{\dagger}) \equiv \partial_t u^{\dagger}(x,t) + \mathcal{D}^{\dagger}(u)u^{\dagger}(x,t) = 0 \quad (x,t) \in \mathcal{T}\times\Omega \tag{1.8}$$

$$u^{\dagger}(x,T) = \partial_u I(u(x,T)) \tag{1.9}$$

$$u^{\dagger}(x,t) = b^{\dagger}(x,t) \quad x \in \Omega \tag{1.10}$$

is judiciously chosen such the last two lines in (1.7) are zero, which furnishes a direct relationship between initial condition perturbations $\delta u(x,0)$ and changes in the cost function $\delta\mathcal{J}$ via

$$\delta\mathcal{J}(u) = \int_{\Omega} u^{\dagger}(x,0)\delta u(x,0)\, d\Omega.$$

Solving (1.8-1.10) for $u^{\dagger}$, the change in cost $\delta\mathcal{J}$ can be found for *any* initial condition perturbation. However, the expression depends on evolving the adjoint solution from time $t = T$ to $t = 0$, which in general depends on the entire state of $u(x,t)$ via $\mathcal{D}^{\dagger}(u)$ in (1.8). For practical applications, accessing $u(x,t)$ can be onerous in terms of memory and computational expense (see **?** for example and discussion), making low-cost low-memory approximations of u (and $u^{\dagger}$) attractive. Thus, the adjoint-forward deep Galerkin method (AFDGM) approximates both u and $u^{\dagger}$ with deep neural networks $f(x,t;\theta)$ and $f^{\dagger}(x,t;\theta^{\dagger})$. The parameters of the neural networks $\Theta = \left\{\theta,\theta^{\dagger}\right\}$ are found by minimizing

$$\begin{aligned}\mathcal{L}(f,f^{\dagger}) = &\underbrace{||\mathcal{M}(f)||^2}_{\mathcal{L}_{PDE}} + \underbrace{||f(\cdot,0) - u_0||^2}_{\mathcal{L}_{IC}} + \underbrace{||f(\hat{x},t) - b(\hat{x},t)||^2}_{\mathcal{L}_{BC}} \\ &+ \underbrace{||\mathcal{M}^{\dagger}(f,f^{\dagger})||^2}_{\mathcal{L}^{\dagger}_{PDE}} + \underbrace{||f^{\dagger}(\cdot,T) - \partial_f I(f(x,T))||^2}_{\mathcal{L}^{\dagger}_{IC}} + \underbrace{||f^{\dagger}(\hat{x},t) - b^{\dagger}(\hat{x},t)||^2}_{\mathcal{L}^{\dagger}_{BC}}\end{aligned} \tag{1.11}$$

where $\hat{x} \in \partial\Omega$. Note the explicit coupling between the approximate forward and adjoint solutions when terms in the PDE and initial conditions are evaluated. The AFDGM algorithm, which is an adaptation of DGM (**?**), is provided in Algorithm 1. An example of the evolution of the loss function and component terms of (1.11) is show in figure 1. Notice, similar trends in $\mathcal{L}_{PDE}$ and $\mathcal{L}^{\dagger}_{IC}$, which shows the coupling between the two neural network models: as u and $u(x,T)$ converges to the true solution so does the initial condition of the adjoint which depends on the forward state $u^{\dagger}(x,T) = \partial_u I(u(x,T))$. The following section provides an example of AFDGM applied to the advection-diffusion equation.

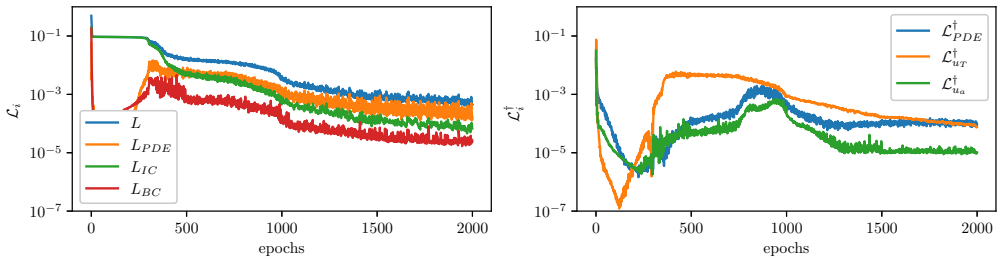

FIGURE 1. Evolution of the loss function components.

Algorithm 1: Algorithm for AFDGM

Input: Number of sampled points in the domain N
Initialize the optimizer learning rate r
Initialize neural network parameters, $\Theta = \{\theta, \theta^\dagger\}$, using the Xavier method
$j = 0$
while Insufficient reduction in objective function **do**
  Generate N random samples $s_n = \{x_n, t_n\}$ in the spatial/temporal domain
  for N_b number of iterations per batch **do**
    Construct the forward-adjoint objective function
    $\mathcal{L} = \mathcal{L}_{PDE} + \mathcal{L}_{BC} + \mathcal{L}_{IC} + \mathcal{L}^\dagger_{PDE} + \mathcal{L}^\dagger_{BC} + \mathcal{L}^\dagger_{TC}$
    Update $\Theta = \{\theta, \theta^\dagger\}$ with stochastic gradient descent step
    $\Theta_{j+1} = \Theta_j - r\nabla_\Theta \mathcal{L}(\Theta)$
    $j \leftarrow j + 1$
  end
  Update learning rate r according to prescribed schedule, e.g., $r \leftarrow r/10^n$
end
Result: Forward and adjoint neural networks: $f(x, t, \mathbf{p}; \theta)$ and $f^\dagger(x, t, \mathbf{p}; \theta^\dagger)$

2. Sensitivity in the advection-diffusion equation

The approach described in the previous section is applied in the context of the linear advection-diffusion system. The objective is to determine what initial condition perturbations $\delta u(x, 0)$ increase the final state energy

$$\mathcal{J}(u) = \frac{1}{2}\int_\Omega u(x,T)^2\, d\Omega. \tag{2.1}$$

The forward system is

$$\frac{\partial u}{\partial t} + c\frac{\partial u}{\partial x} - \nu\frac{\partial^2 u}{\partial x^2} = 0 \tag{2.2}$$

$$u(x, t = 0) = f(x) \quad x \in \Omega, \tag{2.3}$$

where the advection speed is $c = 1.0$ and the diffusion coefficient is $\nu = 0.01$. The initial condition is $f(x) = \exp\left[-(x - 0.5)^2/0.01^2\right]$. The corresponding adjoint system is

$$\frac{\partial u^\dagger}{\partial t} + c\frac{\partial u^\dagger}{\partial x} + \nu\frac{\partial^2 u^\dagger}{\partial x^2} = 0 \tag{2.4}$$

$$u^\dagger(x, t = T) = u(x, t = T) \quad x \in \Omega. \tag{2.5}$$

The boundaries are periodic, e.g. $u(0, t) = u(1, t)$, $\partial_x u(0, t) = \partial_x u(1, t)$, etc. The behavior of the forward and adjoint solutions are shown in figure 2(a,b). The orientation of the

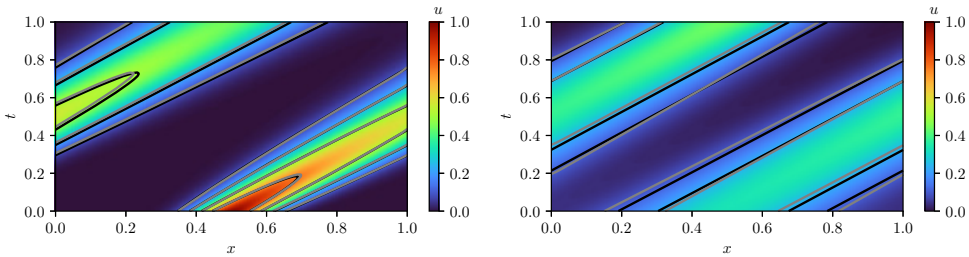

FIGURE 2. Neural network approximation of (a) $u(x,t)$ and (b) $u^\dagger(x,t)$. Colour is $f, f^\dagger$. Contour lines are $\{u, u^\dagger\} = [0.1, 0.25, 0.50.75]$: neural network approximations (black lines) and finite difference discretization (grey lines)

.

contours show that the advection speed is unity. The diffusion term causes the initial Gaussian profile to spread as time advances. The adjoint field, starting from the diffused $u(x,T)$ profile retreats from $t = T$ to $t = 0$ and diffuses further. The final profile in $u^\dagger(x,0)$ shows how to perturb the initial condition $u(x,0)$ in order to increase or decrease $\mathcal{J}$.

The outcomes of two methods are contrasted in figure 2. Gray contour lines solve the forward and adjoint methods using standard finite difference approachs: fourth-order Runge–Kutta time-discretiztaion and second-order central spatial differences. Black contour lines represent the deep network approximations using AFDGM. Slight discrepancies remain due to early stopping of optimizing the neural network parameters, see figure 1. The following section quantifies the accuracy of a cost function $\nabla\mathcal{J}$ for the deep network approximations.

2.1. *Assessment of accuracy*

Often, the accuracy of the gradient is assessed by using Taylor series approximation (**??**)

$$\mathcal{J}(u + \delta u) - \mathcal{J}(u) = \frac{\partial \mathcal{J}}{\partial u}\delta u + O(||\delta u||^2). \tag{2.6}$$

Provided a perturbation δu, the change in the cost can be evaluated $\mathcal{J}(u_0 + \delta u) - \mathcal{J}(u_0)$ by a 'brute-force' manner either using standard discretization schemes or with a neural network approximation of the solution. We select perturbations of the initial condition to be in the direction of the adjoint field, $\delta u_0 = \alpha u_0^\dagger$ where $\alpha > 0$ increases $\mathcal{J}$ and vice versa. The gradient error between the brute-force approach and the adjoint method is

$$\varepsilon = \left| \frac{\mathcal{J}(u_0 + \alpha u_0^\dagger) - \mathcal{J}(u_0)}{\alpha} - {u_0^\dagger}^\top u_0^\dagger \right| + O(\alpha), \tag{2.7}$$

which converges at a rate α. For $|\alpha| \ll 1$, the finite difference $\mathcal{J}(u_0 + \delta u) - \mathcal{J}(u_0)$ is susceptible to numerical round-off and ϵ will increase. Also, when u and $u^\dagger$ are approximated with finite differences from the continuous PDEs, discretization errors impede convergence rates (see **?**). For neural-network approximated forward and adjoint solutions, two errors are assessed:

$$i) \quad \widetilde{\varepsilon} = \left| \frac{\mathcal{J}\left(u_0 + \alpha u_0^\dagger\right) - \mathcal{J}(u_0)}{\alpha} - {f_0^\dagger}^\top f_0^\dagger \right| + O(\alpha) \quad \text{and} \tag{2.8}$$

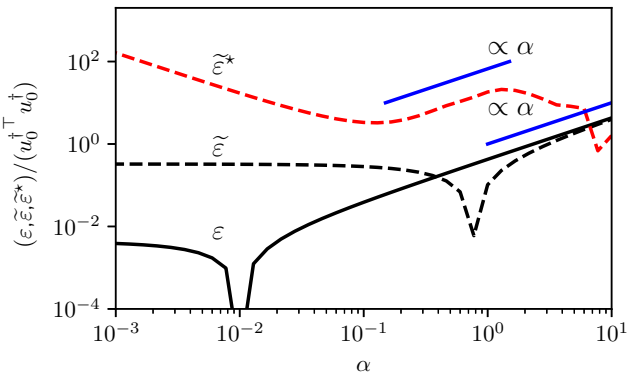

FIGURE 3. Gradient accuracy assessed by perturbing the initial condition with $\delta u_0 = \alpha u^\dagger$.

$$ii) \quad \widetilde{\varepsilon}^\star = \left| \frac{\mathcal{J}\left(f(\alpha f_0^\dagger)\right) - \mathcal{J}(f)}{\alpha} - f_0^{\dagger\top} f_0^\dagger \right| + O(\alpha). \tag{2.9}$$

The $\widetilde{\epsilon}$ compares the neural network gradient to the standard discretization brute-force approach; and, $\widetilde{\epsilon}^\star$ compares the neural network gradient with the forward neural network using a perturbed initial condition $f(\alpha f_0^\dagger)$. Figure 3 shows the relative error. ε is the baseline we expect from solving the discretized forward and adjoint equations with standard discretization schemes. The gradient from the adjoint neural network also shows good agreement with discretized approach. However, when the neural network is used to provide $f(\alpha f_0^\dagger)$, the error is elevated. Comparing $\widetilde{\epsilon}$ and $\widetilde{\epsilon}^\star$, the elevated error is due to computing $f(\alpha f_0^\dagger)$ since the forward network was not trained on perturbations to the initial condition. Thus, the accuracy degrades. The error is further exacerbated by single precision data types used in the training which causes machine round-off to dominate by $\alpha \lesssim 10^{-1}$. In addition, nonlinearity in the neural network is apparent. For $\alpha > 1$, $\widetilde{\epsilon}^\star$ is not $\propto \alpha$.

3. Discussion

Deep learning algorithms provide efficient access to field sensitivity. Gradient accuracy shows an expected rate of convergence, and we expect that more training or deeper networks (more layers and more nodes) can enhance accuracy to a desired level.